Assignment 3

Recommended readings

- Lecture slides as starting literature
- MDN Links within slides for details, especially:
  - setInterval ○ clearInterval ○ replace

Note: All exercises must be solved with plain JavaScript and CSS, not using additional libraries or frameworks.

Exercise 1 – HTML + CSS + JS + DOM: Number-Guessing Game II 0,2 Points

---

This exercise lets you combine all your previous knowledge. Now, you are going to make a more realistic number-guessing game.

Create an HTML file with the following components:

- a header including title, CSS file, and JS file
- a heading stating Number-Guessing Game
- a paragraph with a welcome message to the user
- a fieldset with a number input, with the legend requesting the user to enter a number between 1 and 10
- a disabled button for checking
- a button for starting the game

Create a CSS file with the following components and embed it into the HTML file:

- The heading should be formatted in the font Verdana with a size of 2em and a margin of 15px;
- The paragraph should be centered and have a margin of 10px.
- The fieldset shall be centered.
- The input element should have a “lightblue “ focus color.
- The fieldset should be invisible at first, as should the green disabled checking button be.
- Only the starting game button in the center of the page, in blue, is visible.

Create a JS file that shall provide the following functionality (see next page) and embed it into the HTML file. Use event listeners and DOM operations.

1. **Start Game:**
   - When the "Start Game" button is clicked, the welcome message and the start button should disappear.
   - The game fieldset (containing the number input) should become visible.
   - The "Check" button should become visible in the center of the page and enabled.
   - A dynamically created red "Exit Game" button should be added to the center of the page.
   - A random number between 1 and 10 should be generated.
   - Set the correct attributes of the number input so that only values from 1 to 10 are allowed.

2. **Guessing:**
   - When the user enters a number between 1 and 10 in the input field, the check button gets enabled.
   - When the check button is clicked:
     - The user's guess is compared to the random number.
     - If the guess is correct, a congratulatory message is displayed, and the check button is disabled.
     - If the guess is incorrect:
       - A message indicating whether the guess is "Too high!" or "Too low!" is displayed.
       - The user's guess is added to a list of previous guesses.
       - The list of previous guesses is displayed.
     - The messages and previous guesses shall be displayed between the fieldset and the buttons.
       (Hint: create 2 div elements and just update them dynamically)
   - The user has a maximum of 3 attempts to guess the number.
   - If the user exceeds 3 attempts, the game ends, the correct number is revealed, and the check button is disabled.

3. **Exit Game:**
   - When the “Exit Game” button is clicked, the game is reset.
     - The game fieldset should disappear.
     - The welcome message and “Start Game” button should reappear in the center.
     - All previous guesses are cleared.
     - The elements to show the feedback message and the previous guesses are removed.

Feel free to add any additional styles dynamically.

Exercise 2 – Making an HTML Page Editable 0,3 Points

Given the HTML document *editable.html* in folder *editable*, with sections and one h1 heading element as a child of each section. Write a JavaScript file *editable.js* fulfilling the following requirements when referenced from the HTML document:

- Your code adds a form containing a select box, a text input, a textarea, and an initially disabled submit button after the last section.
- The select box allows the user to select any section of the document based on the heading of the section. After selection, the text input element shows the heading of the selected section, the textarea shows the innerHTML of the selected section, and the button is activated.
- When the button is clicked, the content of the selected section is updated with the data of the input element and the textarea. If the heading is changed in the textarea and in the input field, only the value of the input field applies. Be sure to correctly handle cases where the user removes the heading element in the HTML source. Also, be sure to update the content of the select box after a heading has been changed.

Do not change the HTML document except by adding the reference to your JavaScript file!

Exercise 3 – Image Gallery Part 2 0,2 Points

Given the picture gallery from Assignment 2 exercise 6. Extend the gallery such that when a picture is clicked and shown in full window mode, the user has controls for:

- "*play/pause*", implementing an automatic slideshow showing the pictures in an endless loop until *pause* or *exit* is clicked. The duration for showing each picture should be equal to the number of seconds specified in an input field next to the buttons (default value: 3 seconds).
- "*exit*", for stopping an eventually running slideshow and closing the full window mode.

Important: Your script must work with any HTML content following the format of the HTML document of the lecture.

Exercise 4 – Puzzle Game in JavaScript 0,3 Points

The folder *puzzle* contains the HTML document *puzzle.html*. The page shows a grid with 4 fields and a list of 4 images. The goal of the game is to move each image in the list to the correct grid cell to solve the puzzle.

Moving images should be realized as in the lecture slides. If the user puts an image onto a field in the grid and clicks (the mouse position is the relevant position), then the image is removed and set as the background of the specified grid cell. If there is already another picture in the grid cell, then the previous picture is moved back to the list of pictures. When all pictures are in the correct position of the grid, the user wins, and a message is shown; otherwise, a message of wrong placement is shown.

Note: The "grid" is actually realized by the position of some divs. The solution is encoded in the data attribute *"data-result"* of each div. Your solution should not be restricted to the grid. It should be sufficient that each picture is moved to the div with the matching data-result attribute.

While you should apply the general principle of the lecture for moving the images to the grid, you can change the code so that the mouse is not positioned above the currently moved image while moving. This makes it easier to interact with the grid.

Recommended Project Preparations

Consider everything you learned in this assignment for planning where you will need eventListeners in order to update, add, or delete elements.